

Занятие №2

Статистический анализ и
визуализация данных

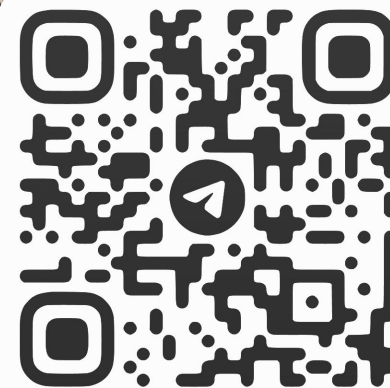
октябрь 2025



Дмитрий Кротов

И.о. Team Lead аналитиков опыта исполнителя
в Авито Подработка, аспирант.

- Alma mater: Финансовый университет при Правительстве РФ
- 5 лет в Аналитике (ex Avito Авто, Ozon)
- Преподаватель во внешней и внутренней Академии Аналитиков Авито
- Основатель и автор тг-канала Data dreamen
- Автор тг-канала Авито «Коммуналка аналитиков»



@DATA_DREAMEN

Как мы общаемся на занятии

Вопросы по теме занятия пишите в чат, начиная сообщение со слова «Вопрос:....»

Поднимайте руку, если хотите задать вопрос голосом или развернуто

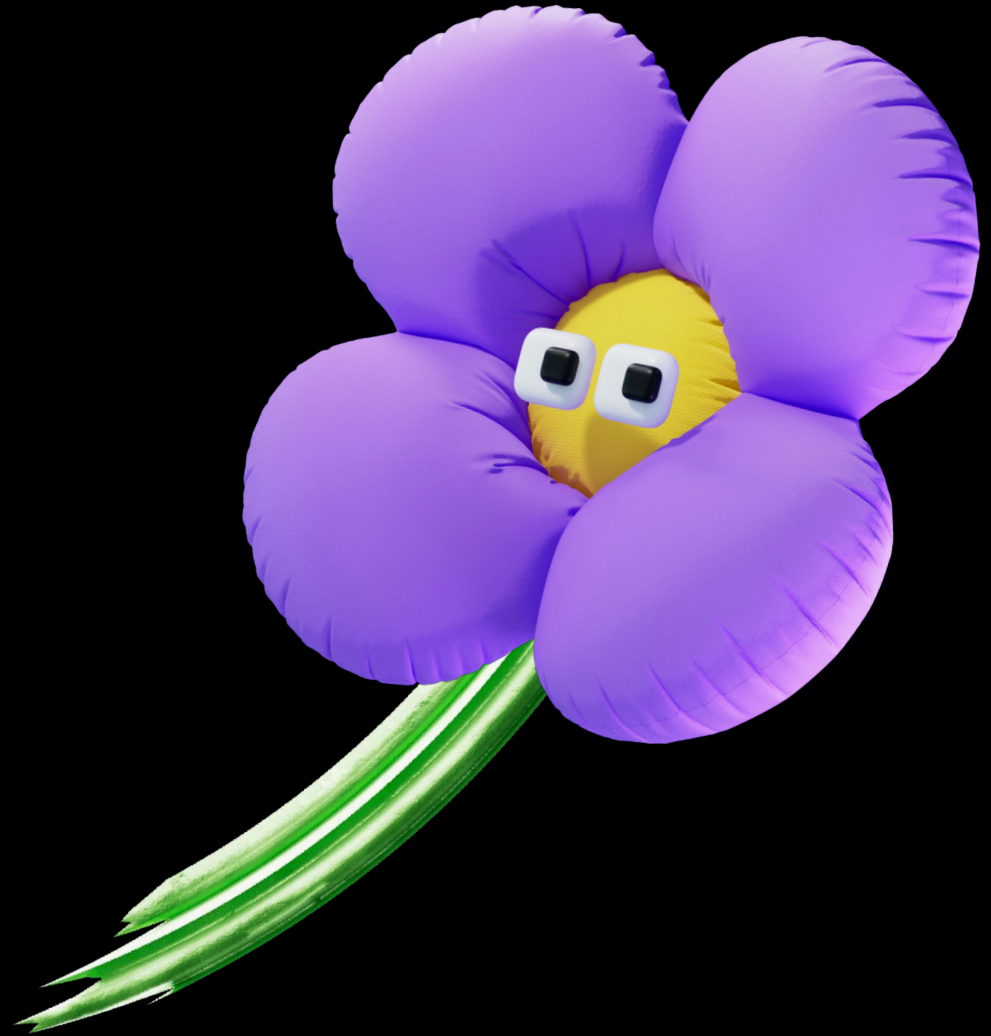
Организационные вопросы по итогам курса и расписанию задавайте в Telegram-чате

Поддерживаем уважение друг к другу и эксперту

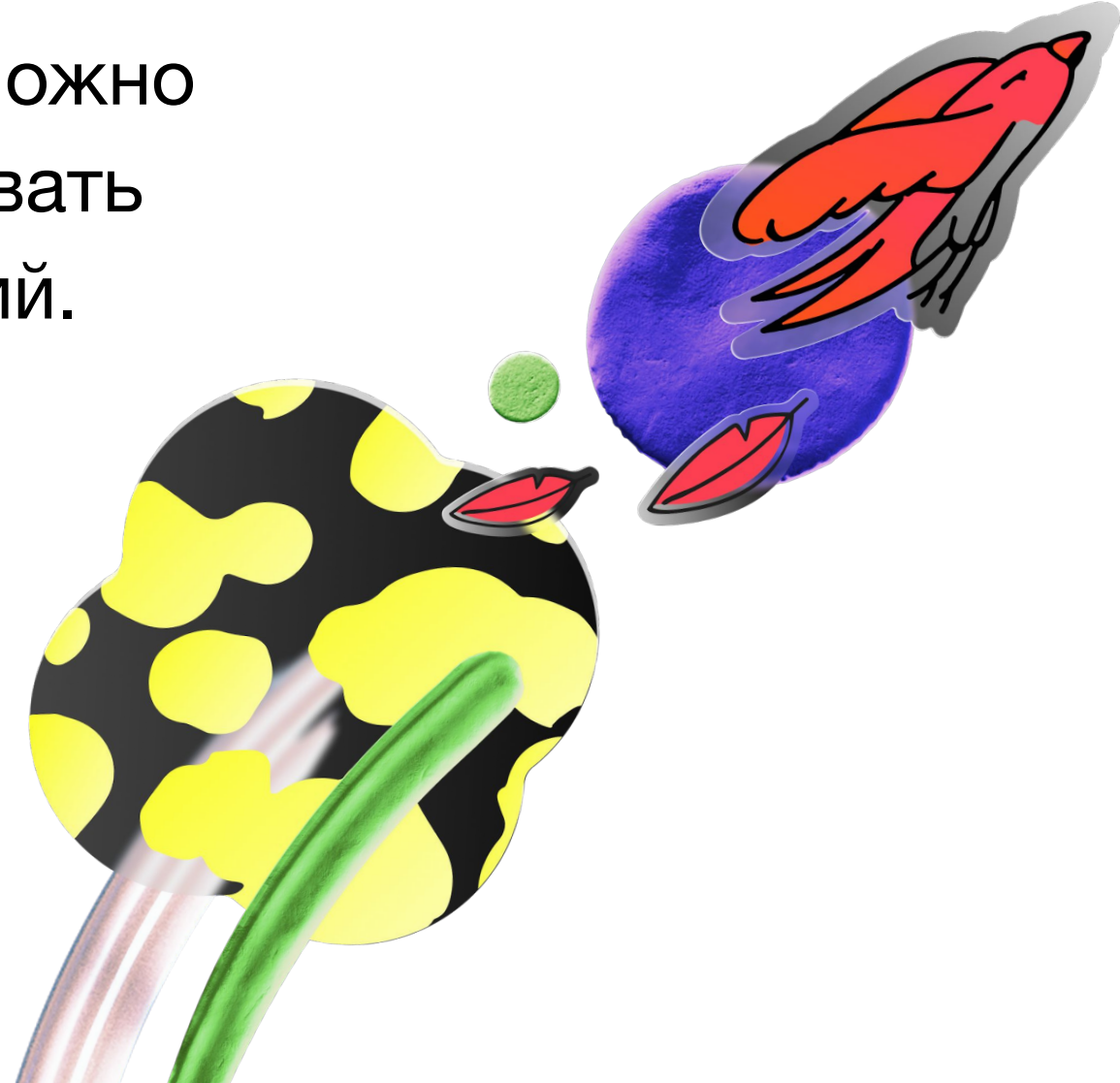




Типы данных, источники, цели и задачи EDA



Данные — это зафиксированная информация о мире, которую можно собрать, сохранить и использовать для анализа и принятия решений.



Что такое данные?



Данные

Сырые факты, цифры, символы, события.

Ещё не имеют смысла без контекста.



Информация

Данные, приведённые в систему и осмысленные.

Отвечают на вопрос: «Что это значит?»



Знания

Интерпретация информации с учётом опыта, закономерностей, контекста.

Отвечают на вопрос: «Почему это так?» и «Что из этого следует?»



Решения

Применение знаний для действий и стратегий.

Отвечают на вопрос: «Что делать?»

Типы данных

Числовые

Непрерывные

Могут принимать любые значения в диапазоне.

Примеры: цена товара, расстояние до пользователя, время до первой сделки.

Дискретные

Ограниченные целые значения.

Примеры: количество фотографий в объявлении, число просмотров, число сообщений.

Категориальные

Номинальные

Категории без порядка.

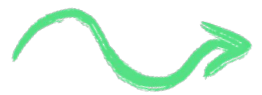
Примеры: тип товара («телефон», «велосипед», «диван»), регион, способ доставки.

Порядковые

Категории с естественным порядком.

Примеры: состояние («новый», «хороший», «средний»), уровень аккаунта («базовый», «премиум»).

Типы данных



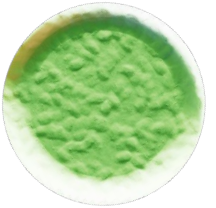
Текст: заголовки, описания, отзывы, сообщения.



Изображения/видео: фото товара, видеообъявления.



Аудио (редко, но возможно): звонки, голосовые сообщения.



Разведочный анализ данных (EDA)

Разведочный анализ данных (EDA) — системный способ понять структуру, качество и «поведение» данных до моделирования.

Главная цель — понять, что представляют собой данные, выявить их особенности и ограничения, чтобы можно было:

- сформулировать корректные гипотезы;
- выбрать подходящие методы анализа/моделирования;
- снизить риски ошибок и неверных выводов.



Задачи EDA

Понимание структуры данных

- Какие есть таблицы, поля, типы признаков.
- Проверка первичных ключей, уникальности и связей.

Изучение распределений

- Средние, медианы, квантили, хвосты.
- Поиск выбросов и экстремальных значений.

Работа с пропусками и дубликатами

- Определение доли и паттернов пропусков.
- Поиск дублированных записей.

Анализ временных и геоданных

- Сезонность, тренды, аномалии.
- Географические распределения и кластеры.

Анализ связей между признаками

- Корреляции между числовыми признаками.
- Зависимости категориальных признаков.
- Первые гипотезы о факторах, влияющих на целевую переменную.

Выявление проблем качества данных

- Ошибки логирования, несоответствия форматов, неверные единицы измерения.
- Потенциальные источники смещений (bias) и утечки таргета.

Формулировка гипотез и идей для моделей

- Какие признаки могут оказаться полезными.
- Какую трансформацию стоит попробовать.
- Какие сегменты пользователей различаются по поведению.

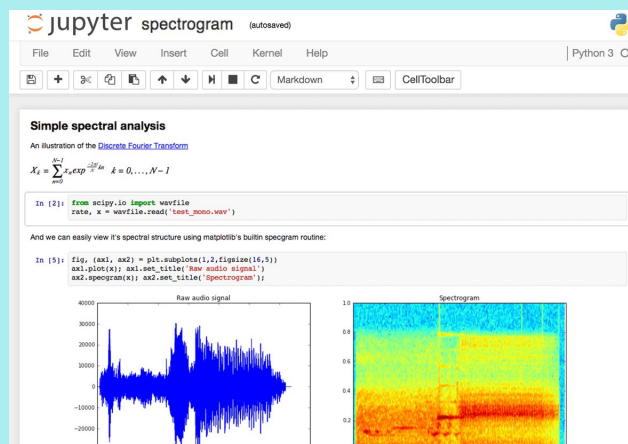
Инструменты EDA

SQL

```
ALTER VIEW sales_by_film_category
AS
SELECT
c.name AS category
, max(p.amount) AS max_sales
FROM payment AS p
INNER JOIN rental AS r ON p.rental_id = r.rental_id
INNER JOIN inventory AS i ON r.inventory_id = i.inventory_id
INNER JOIN film AS f ON i.film_id = f.film_id
INNER JOIN film_category AS fc ON f.film_id = fc.film_id
INNER JOIN category AS c ON fc.category_id = c.category_id
GROUP BY c.name GO
```

Первые выборки, агрегации,
проверка схемы и ключей.

Python



Обработка и статистика
по данным.

Tableau, Power BI, Superset (и др. BI системы)

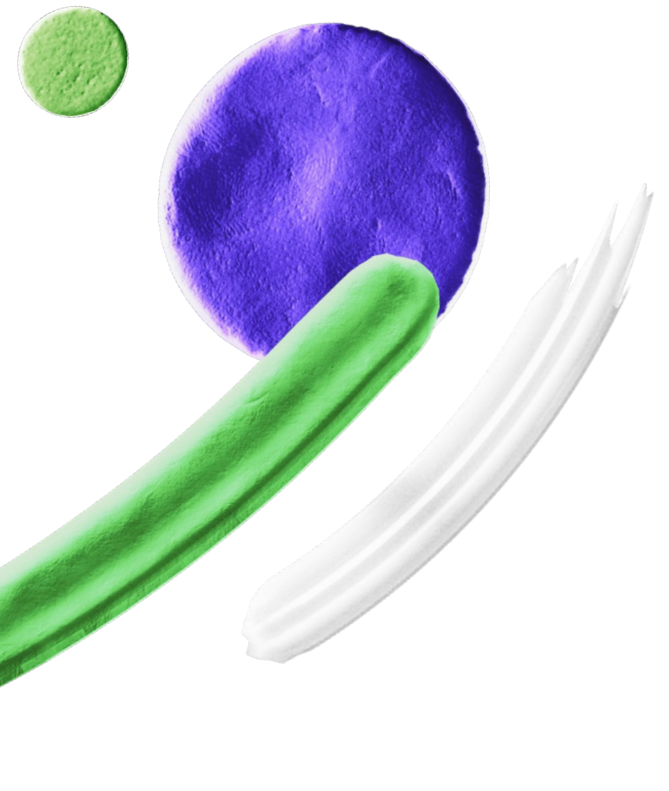


Интерактивные дашборды
для быстрого анализа.

Кейсы влияния EDA на продукт/бизнес

Мы выявили, что каждая десятая загрузка медицинской книжки происходит до верификации личности через Госуслуги, что приводит к дополнительным ошибкам при использовании сервиса.

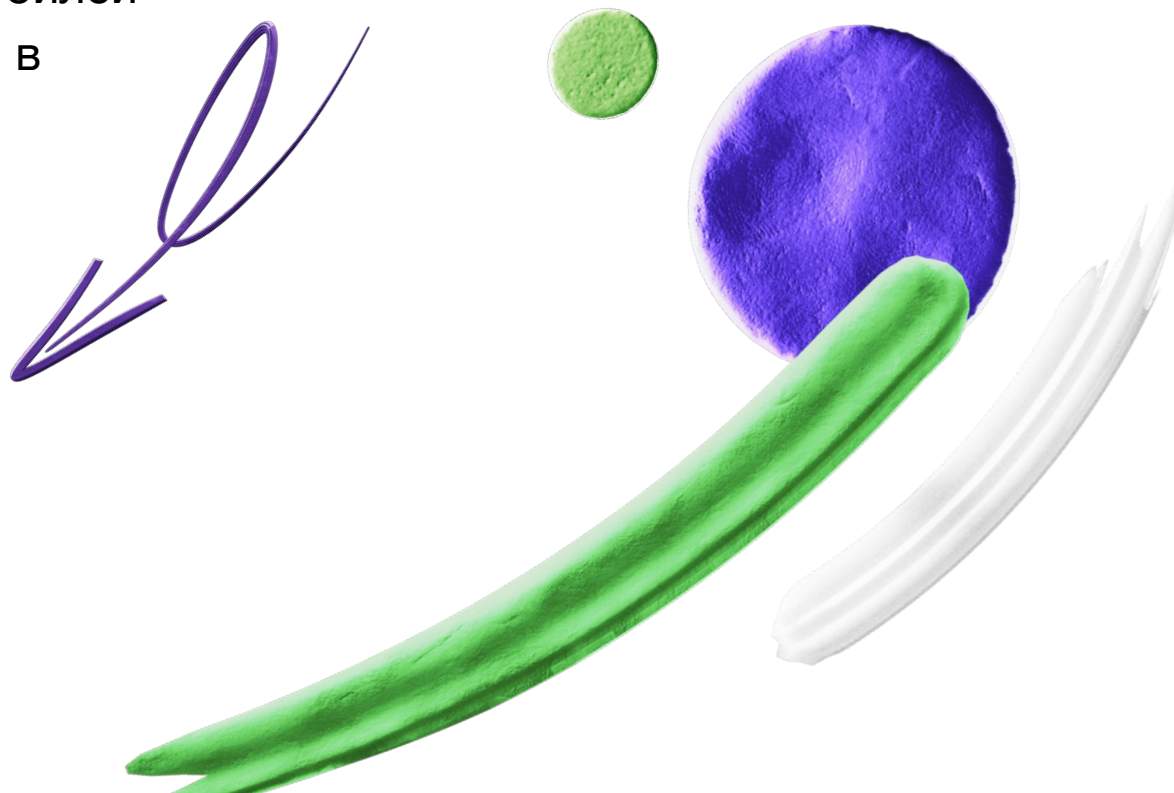
Решение: пошли в пересбор CJM.



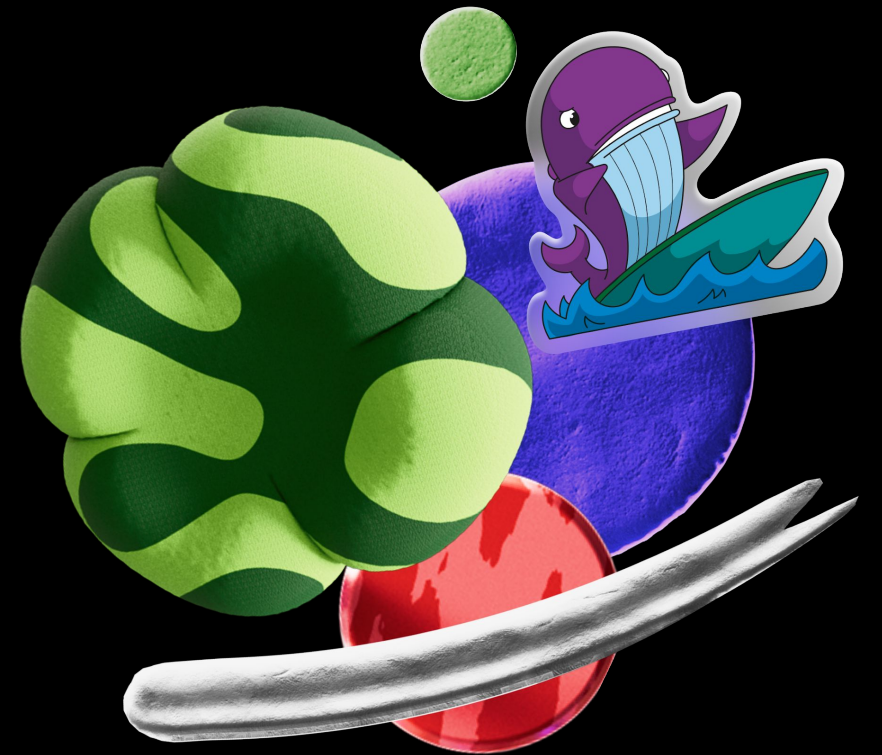
Кейсы влияния EDA на продукт/бизнес

Анализ массива объявлений для рекламы в Директе показал, что большинство фотографий автомобилей имеют низкое качество и отображают машины в плохом состоянии.

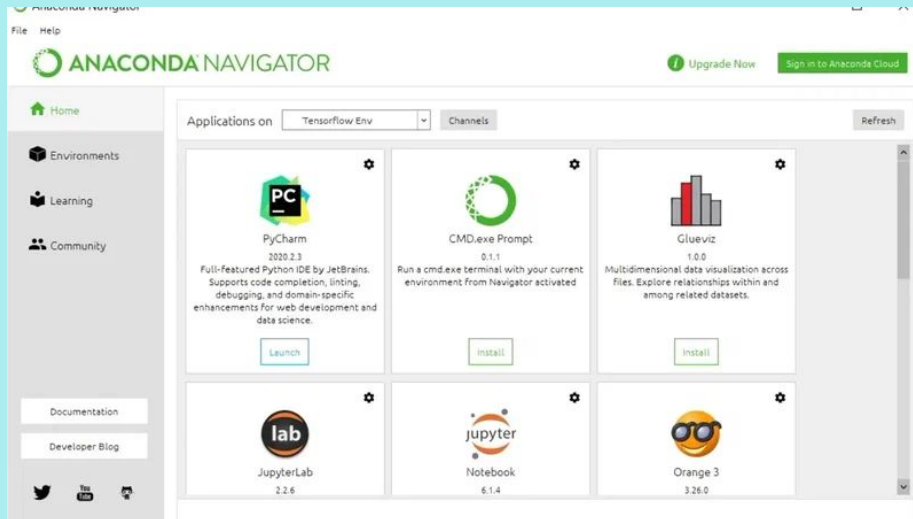
Решение: разработали модель, оценивающую качество автомобиля на фото.



Практическая работа с данными



Инструменты



Jupyter — интерактивный блокнот, первоначально веб-реализация и развитие IPython, ставший самостоятельным проектом.

Скачать можно по ссылке:
<https://anaconda.org/main/jupyter>



Google Colab (Google Collaboratory) — **бесплатный облачный сервис для работы с кодом на Python** прямо в браузере. Основан на технологии Jupyter Notebook, но работает в облаке Google, не требуя установки дополнительного программного обеспечения на локальный компьютер.

Скачать можно по ссылке:
<https://colab.google/>

Pandas

Что такое Pandas

- Библиотека Python для табличных данных (DataFrame/Series).
- Базовые инструмент для загрузки, очистки, трансформаций и базовой аналитики.

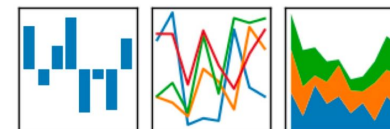
Ключевые возможности

- I/O: чтение/запись CSV, Parquet, Excel, SQL.
- Фильтрации/срезы, сортировки, группировки и агрегаты.
- Джойны/мерджи, сводные таблицы (pivot/pivot_table).
- Работа с пропусками, датами/временем, категориальными признаками.
- Векторные операции и метод-цепочки для воспроизводимости.



pandas

$$y_{it} = \beta' x_{it} + \mu_i + \epsilon_{it}$$



```
import pandas as pd
```


NumPy

Что такое NumPy

- Базовая библиотека Python для работы с многомерными массивами (`ndarray`) и быстрых численных операций

Когда использовать в EDA

- Быстрые агрегаты и трансформации без накладных расходов DataFrame.
- Векторные расчёты, маскирование, матричная алгебра, работа с распределениями.
- Подготовка массивов/фич для моделей, где важна скорость и контроль



NumPy

```
import numpy as np
x = np.array(df["price"], dtype=np.float64)
mask = (x > 0) & (x < np.percentile(x, 99))
x_trim = x[mask]
np.median(x_trim), np.percentile(x_trim, [5, 95])
```

SQLAlchemy

- Универсальный слой работы с БД (PostgreSQL, MySQL/MariaDB, SQLite и др.).
- Два подхода: **Core (SQL-выражения)** и **ORM (модели классов)**.
- Основа для EDA в компаниях с DWH: единый способ подключаться, запускать запросы и тянуть данные в Pandas.



```
from sqlalchemy import create_engine

# PostgreSQL
engine = create_engine("postgresql+psycopg2://user:pass@host:5432/dbname")
```

```
q = """
SELECT category, COUNT(*) AS n,
       PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY price) AS p50
FROM ads
WHERE status = 'active'
GROUP BY category
ORDER BY p50 DESC
"""

df = pd.read_sql(q, con=engine)
```

PySpark

PySpark — это Python API для Apache Spark, инструмент для обработки больших данных и аналитики. Позволяет использовать возможности распределённых вычислений Spark, используя знакомый язык программирования Python.



Matplotlib / Plotly

Что это

- **Matplotlib:** базовая графика, полный контроль.
- **Plotly:** интерактивные графики в ноутбуке/вебе.

Зачем в EDA

- Быстро увидеть распределения, хвосты, связи, сезонность.

matplotlib

plotly

```
import matplotlib.pyplot as plt
df["price"].plot(kind="hist", bins=50); plt.title("Price distribution"); plt.show()
```

```
import plotly.express as px
px.scatter(df, x="price", y="views", trendline="ols")
```

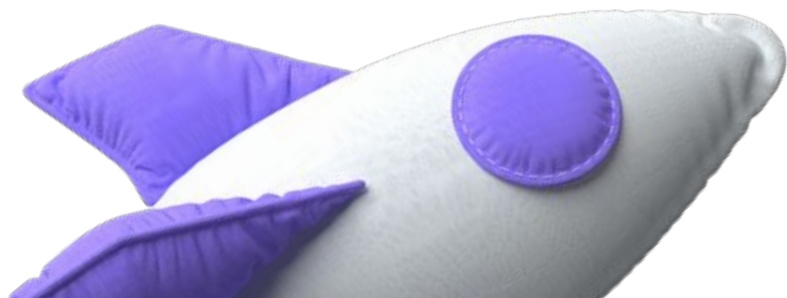
Seaborn

Seaborn — библиотека для создания статистических графиков. Построена на основе библиотеки Matplotlib и тесно интегрирована со структурами данных из Pandas



```
import seaborn as sns
sns.set_theme(style="whitegrid") # аккуратные дефолты

sns.scatterplot(data=df, x="price", y="views", hue="category", alpha=0.6)
```



Уровень абстракции:

- *Matplotlib* — «низкоуровневая» графика (полный контроль над осями/художником).
- *Seaborn* — «высокоуровневые» функции (plot = тип данных + семантики).

Работа с данными:

- *Matplotlib* принимает массивы/серии.
- *Seaborn* понимает **DataFrame** + имена столбцов, **tidy data** (x, y, hue, col, row).

Статистика из коробки:

- *Matplotlib* — без статистики.
- *Seaborn* — регрессии, доверительные интервалы, KDE, категориальные эстиматоры.

Гриды и фасеты:

- *Seaborn* даёт **FacetGrid/PairGrid** для панелей по срезам.

Стиль:

- *Seaborn* сразу даёт приятные темы/палитры;
- В *Matplotlib* надо вручную настраивать стиль и цвета.

SciPy / Statsmodels

Что это

Набор статистических функций и моделей.

Зачем в EDA

Проверка распределений, корреляции, значимости различий, регрессии.



```
import statsmodels.api as sm
X = sm.add_constant(df[["x1", "x2"]]); y = df["y"]
sm.OLS(y, X).fit().summary()
```

```
from scipy import stats
stats.mannwhitneyu(df[df.grp==0]["price"], df[df.grp==1]["price"], alternative="two-sided")
```

Scikit-learn

Что это

- Набор методов ML; в EDA помогает быстро проверять гипотезы.

Зачем в EDA

- Стратифицированные сплиты, трансформеры, базовая важность признаков.



```
from sklearn.model_selection import train_test_split
from sklearn.metrics import roc_auc_score
from sklearn.ensemble import RandomForestClassifier

X = df[features]; y = df["target"]
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.2, stratify=y, random_state=42)
mdl = RandomForestClassifier(n_estimators=200, random_state=42).fit(Xtr, ytr)
roc_auc_score(yte, mdl.predict_proba(Xte)[: ,1])
```


Как скачать библиотеку?

Установка библиотек через jupyterhub и collab:

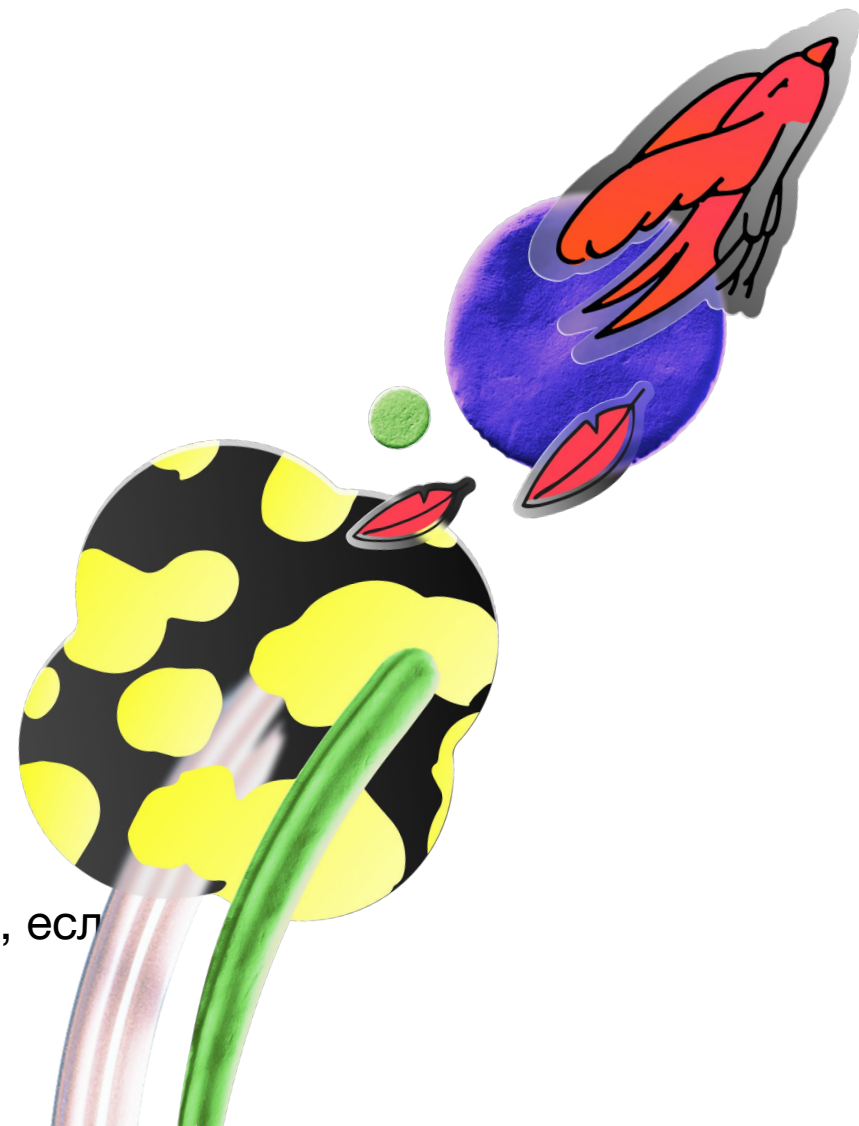
```
# установить пакет
!pip install pandas

# установить конкретную версию / обновить
!pip install "pandas==2.2.3" --upgrade

# несколько пакетов разом
!pip install numpy matplotlib seaborn

# если нет прав на систему
!pip install --user polars
```

После установки **можно перезапустить Kernel** (Kernel → Restart), если импорт не сработал сразу.



Структура 2го занятия

Тема 1. Разведочный анализ данных

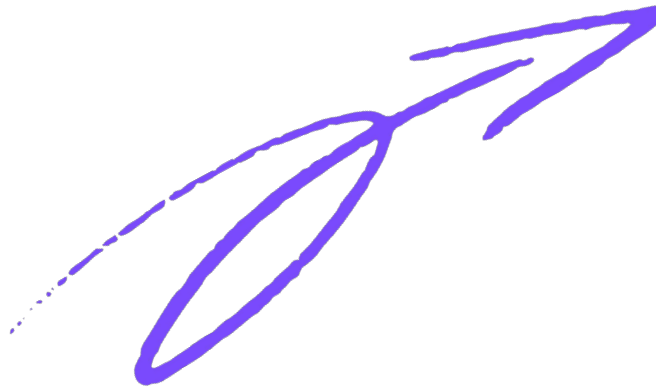
- Введение в работу с данными и разведочный анализ

 ● Цели и задачи и основы EDA

- Углубленный EDA: статистический анализ, визуализация данных, аномалии и текстовые данные

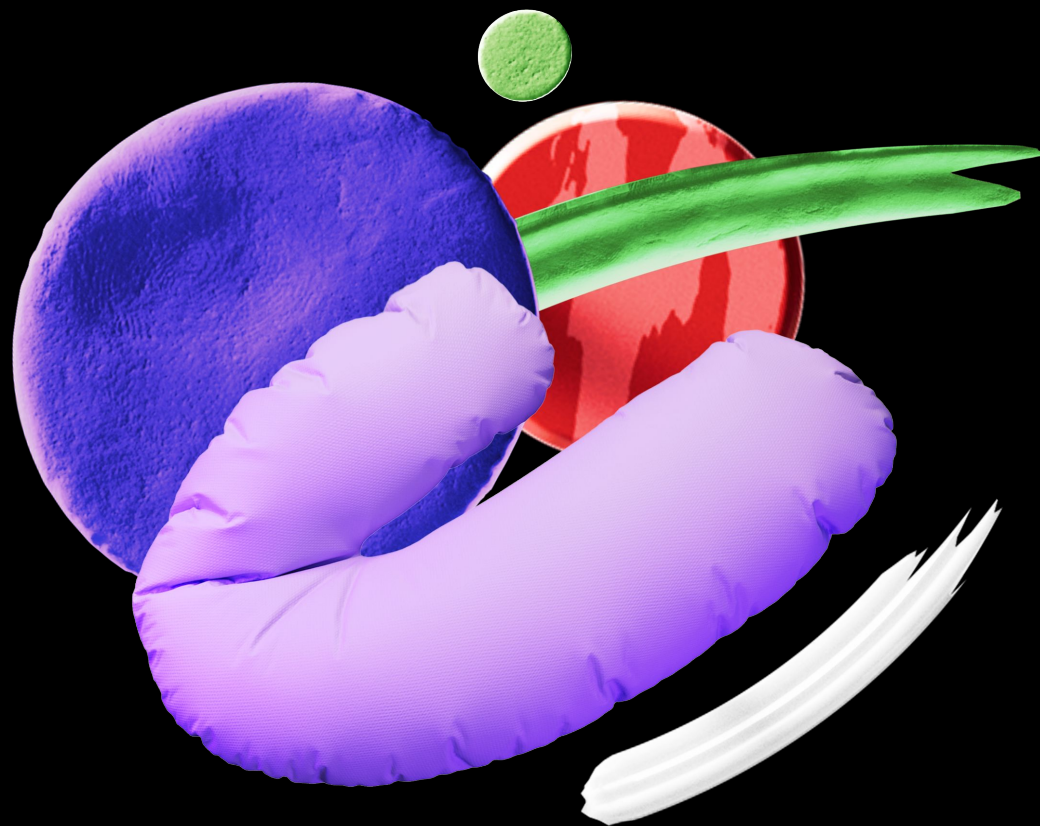
Тема 2. Очистка и подготовка датасета

- Очистка и подготовка датасета к моделям и исследованиям



Первичная проверка данных

Цель: убедиться, что данные
корректно загружены, структура
понятна и нет явных проблем
(первый шаг любого EDA)



Первичная проверка данных

Обзор данных

Проверка и удаление дублей

Корректность данных

Проверка на пропуски
и избавление от них

Проверка форматов

Метаданные

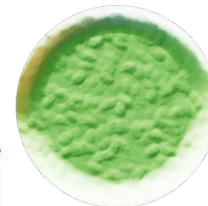
Обзор данных

	S.No.	Name	Location	Year	Kilometers_Driven	Fuel_Type	Transmission	Owner_Type	Mileage	Engine	Power	Seats	New_Price	Price
0	0	Maruti Wagon R LXI CNG	Mumbai	2010	72000	CNG	Manual	First	26.6 km/kg	998 CC	58.16 bhp	5.0	NaN	1.75
1	1	Hyundai Creta 1.6 CRDi SX Option	Pune	2015	41000	Diesel	Manual	First	19.67 kmpl	1582 CC	126.2 bhp	5.0	NaN	12.50
2	2	Honda Jazz V	Chennai	2011	46000	Petrol	Manual	First	18.2 kmpl	1199 CC	88.7 bhp	5.0	8.61 Lakh	4.50
3	3	Maruti Ertiga VDI	Chennai	2012	87000	Diesel	Manual	First	20.77 kmpl	1248 CC	88.76 bhp	7.0	NaN	6.00
4	4	Audi A4 New 2.0 TDI Multitronic	Coimbatore	2013	40670	Diesel	Automatic	Second	15.2 kmpl	1968 CC	140.8 bhp	5.0	NaN	17.74

Как открыть CSV/Excel (`pd.read_csv()`,
`pd.read_excel()`)

Как проверить размер:
`df.shape`, `df.head()`, `df.tail()`

делали на предыдущем занятии



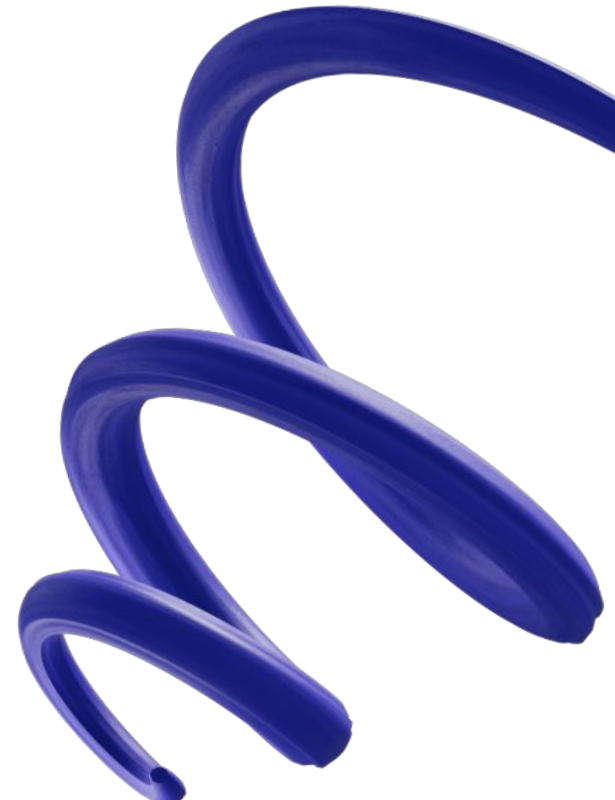
Корректность данных

Проверка диапазонов: `df.describe()` — нет ли отрицательных цен, странных возрастов и т.п.

Типичные аномалии:

- некорректные форматы дат
- перепутанные единицы измерения
- одинаковые значения в разных регистрах (“Москва” / “москва”)

Категориальные признаки: `df['col'].unique()` — увидеть, что “male”, “Male”, “M” не дублируются.

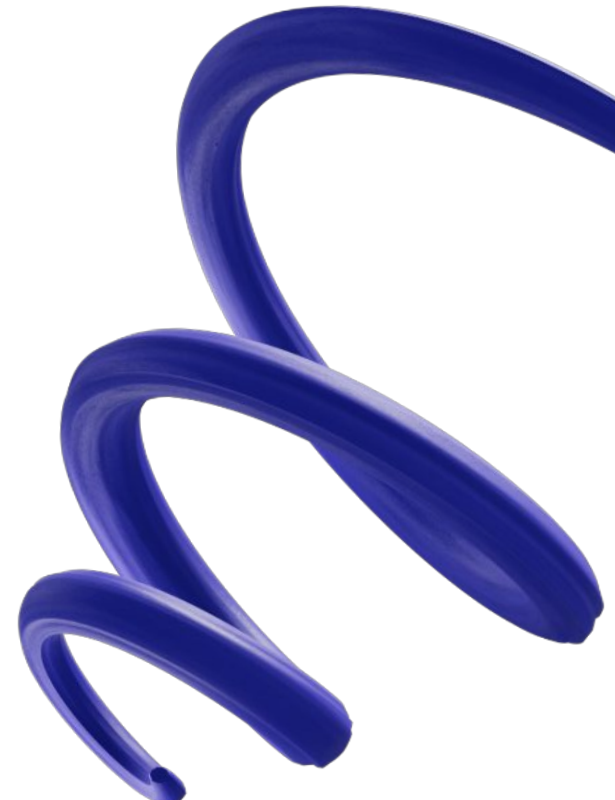


Проверка форматов

In [6]: `df.info()`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7253 entries, 0 to 7252
Data columns (total 14 columns):
 #   Column                Non-Null Count  Dtype  
---  -
 0   S.No.                 7253 non-null   int64  
 1   Name                  7253 non-null   object  
 2   Location              7253 non-null   object  
 3   Year                  7253 non-null   int64  
 4   Kilometers_Driven     7253 non-null   int64  
 5   Fuel_Type             7253 non-null   object  
 6   Transmission          7253 non-null   object  
 7   Owner_Type            7253 non-null   object  
 8   Mileage               7251 non-null   object  
 9   Engine                7207 non-null   object  
10   Power                 7207 non-null   object  
11   Seats                 7200 non-null   float64 
12   New_Price             1006 non-null   object  
13   Price                 6019 non-null   float64 
dtypes: float64(2), int64(3), object(9)
memory usage: 793.4+ KB
```

Определить типы столбцов (`df.info()`),
убедиться, что числа не строками.



Проверка дублей

```
# 1. Проверить, есть ли полные дубликаты строк
df.duplicated().sum()

# 2. Посмотреть сами дубликаты
df[df.duplicated()]

# 3. Проверить дубли по конкретным столбцам (например, ID)
df[df.duplicated(subset=['user_id'])]
```

Как появляются дубли

- Повторная загрузка того же файла.
- Ошибки при сборе данных (два источника дают одно и то же).
- Разные ID для одного объекта (регистры, пробелы, формат даты).
- Некорректные merge / concat без удаления дублей.

Проблема	Что происходит	Последствия
Искажение статистики	значения повторяются → увеличиваются суммы, средние, частоты	неверные метрики, завышенные продажи, просмотры и т. д.
Повторный учет объектов	одна сущность (пользователь, заказ, объявление) встречается несколько раз	дублируются записи, аналитика “считает дважды”
Искажение обучения моделей	алгоритм видит одни и те же наблюдения как независимые	переобучение, смещение весов
Ошибки при объединении таблиц	join с дубликатами создаёт взрывы строк (“картезианское умножение”)	лавинообразное увеличение объёма данных

Как удалить дубли

Перед удалением — посмотрите, почему есть дубли.

Иногда они — не ошибка, а разные события с одинаковыми признаками.
(например, два заказа от одного клиента в один день.)

Добавьте «уникальный ключ» для контроля.

Например, `order_id`, `user_id` + `date`, `ad_id`.

Проверяйте после объединений (`merge`, `concat`).

Особенно если `join` по неполному набору ключей.

Логируйте, сколько дублей удалили.

Это хороший признак качества данных.

Будьте осторожны с временными метками.

Иногда строки кажутся дублями, но отличаются временем на секунды.

1. Удалить полные дубликаты (все столбцы)

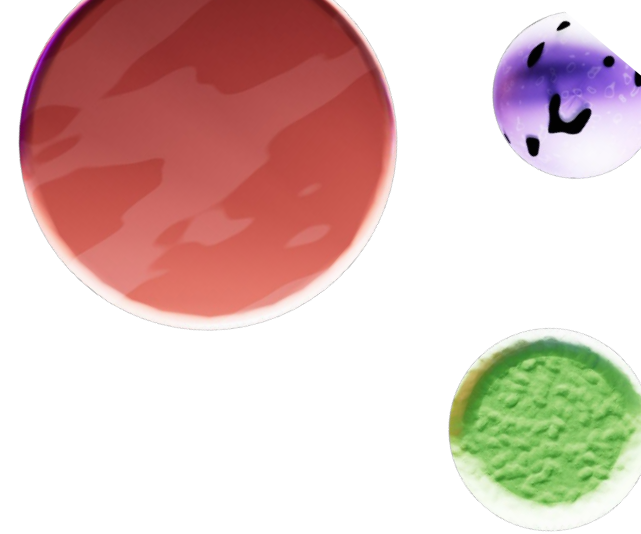
```
df = df.drop_duplicates()
```

2. Удалить по определённым признакам (например, ID)

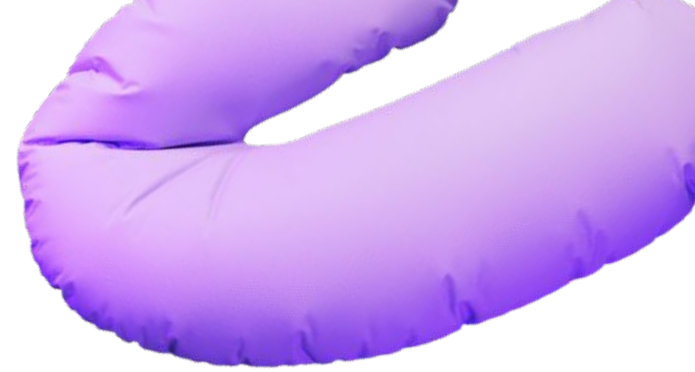
```
df = df.drop_duplicates(subset=['user_id'])
```

3. Если нужно оставить последний дубль

```
df = df.drop_duplicates(subset=['user_id'], keep='last')
```



Проверка на пропуски



Откуда берутся пропуски

- Источник данных не записал значение (человек не заполнил поле, сенсор не сработал).
- Ошибки при сборе/парсинге (например, неверный формат числа).
- Разные источники → разные наборы полей.
- Условные значения («не применимо», «неизвестно», пустая строка).
- После обработки (join, фильтрация, pivot) появились новые NaN.

Почему пропуски в данных опасны

Проблема	Что происходит	Последствия
Искажение статистики	метрики (средние, медианы, корреляции) считаются без части данных	неверные оценки, смещённые выводы
Потеря информации	часть наблюдений “выпадает” из анализа	уменьшается выборка, падает статистическая мощность
Ошибки визуализации	графики искажаются или не строятся	неполная картина данных
Проблемы при обучении моделей	большинство алгоритмов не умеют работать с NaN	ошибки выполнения, смещение моделей
Ошибки при объединении данных	join’ы с NaN-ключами теряют строки	неполные результаты, пропавшие связи

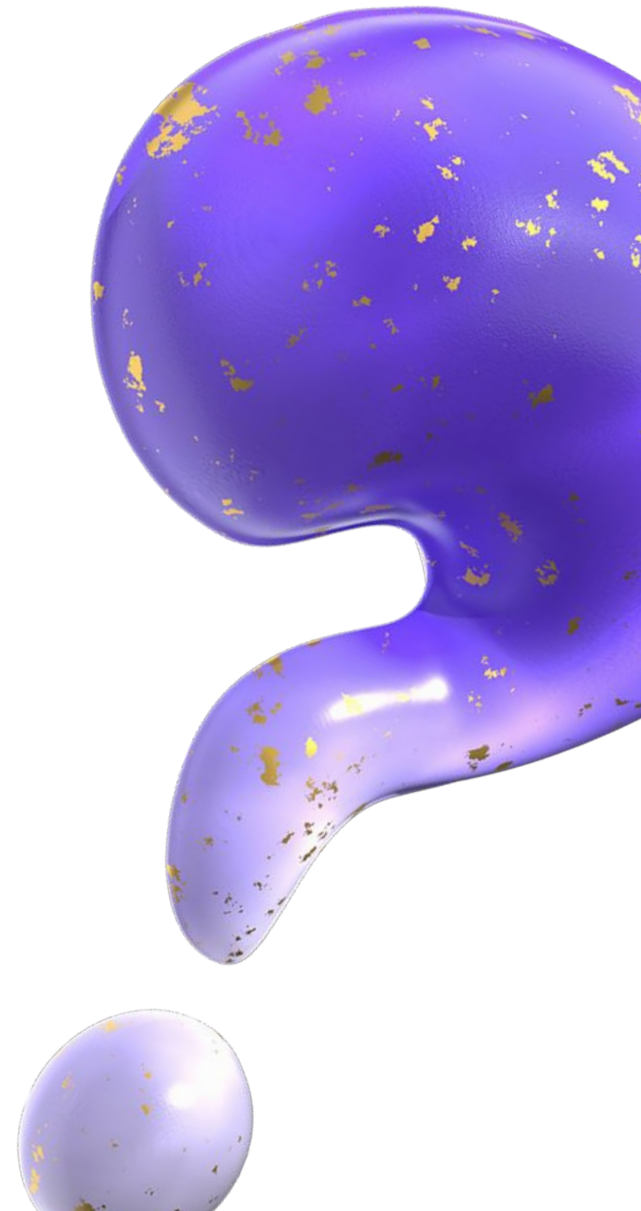
Проверка на пропуски

```
import pandas as pd

# 1. Проверить общее количество пропусков
df.isna().sum()

# 2. Посмотреть долю пропусков
df.isna().mean() * 100

# 3. Визуализировать пропуски (тепловая карта)
import seaborn as sns
sns.heatmap(df.isna(), cbar=False)
```



Проверка на пропуски

1. Удалить пропуски (жёстко)

```
df = df.dropna() # удалить строки с любыми NaN
df = df.dropna(subset=['column']) # удалить, если пропуск в конкретной колонке
```

2. Заполнить (импутация)

```
df['age'] = df['age'].fillna(df['age'].median()) # медианой
df['city'] = df['city'].fillna('Unknown') # категорией
```



Подходит, если доля пропусков мала (<5%)



Не рекомендуется при больших потерях данных.

Тип данных	Чем заполнять	Почему
Числовые	среднее, медиана, 0	сохраняет структуру
Категориальные	"Unknown", мода	не искажает категорию
Временные	<code>ffill()</code> или <code>bfill()</code>	логично для временных рядов
Продвинутые	KNN, регрессия, MICE	если пропусков много и важно качество

Избавление от пропусков

3. Интерполяция (для временных рядов)

```
df['temperature'] = df['temperature'].interpolate()
```

Заполняет значения по соседним точкам (линейно или сплайном). Хорошо работает для постепенных изменений (погода, трафик, продажи).



4. Отметить флагом

```
df['is_price_missing'] = df['price'].isna().astype(int)
```

Позволяет модели «учиться» на факте отсутствия данных.



Виды пропусков

Тип пропуска	Что значит	Пример
MCAR (Missing Completely at Random)	случайные пропуски, не зависят от данных	сенсор иногда не сработал
MAR (Missing at Random)	пропуск связан с другими признаками	доход пропущен чаще у молодых
MNAR (Missing Not at Random)	значение пропущено <i>из-за самого значения</i>	человек не указал вес, потому что он большой

- MCAR можно безопасно игнорировать,
- MAR — можно корректировать статистикой
- MNAR — требует внимательного анализа контекста

Виды пропусков

Сначала исследование, потом заполнение.

Пропуски могут не быть случайными, а показывать закономерность (например, «люди без дохода чаще не отвечают»).

Аккуратное заполнение..

Неправильная импутация хуже самого пропуска — может исказить зависимости.

Сохранение флага пропуска.

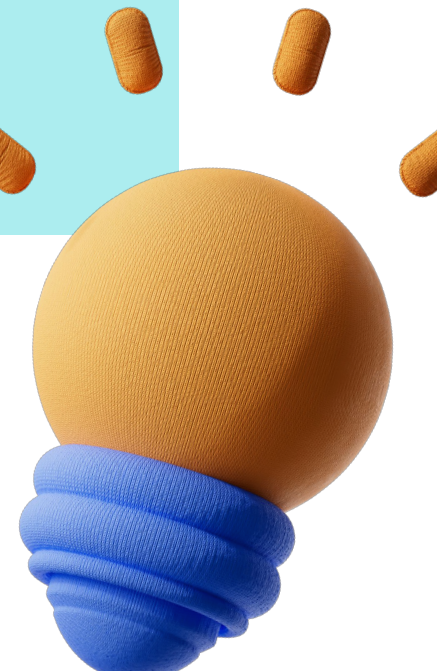
Полезно при моделировании (модель сама может “понять”, где не было данных).

Визуализация пропусков.

Например, `sns.heatmap(df.isna())` или `missingno.matrix(df)` помогает увидеть паттерны.

Анализ пропусков как источник инсайта.

Часто “где пропущено” — уже сигнал о поведении (например, не указал телефон — не хочет контактировать).



Метаданные

Метаданные — это «**данные о данных**», то есть информация, которая описывает структуру, смысл, источники и ограничения самих данных.

Если таблица — это **контент**, то метаданные — это **инструкция, как с ней работать**.

Ценность сохранения и обмена метаданными

Причина	Объяснение
Понимание контекста	Помогает понять смысл признаков и интерпретировать графики правильно.
Правильный выбор методов анализа	Нельзя считать среднее по строковому признаку или корреляцию по категориальному.
Контроль качества данных	Метаданные помогают увидеть ошибки типов, диапазонов и пропусков.
Упрощают объединение источников	Если все поля имеют паспорт (тип, единицы, формат) — merge/concat проще и безопаснее.
Передача знаний	Другой аналитик или студент быстро понимает структуру набора.
Подготовка к моделированию	Позволяет заранее определить, какие признаки использовать, какие закодировать, а какие удалить.

Что входит в метаданные?

Общая информация о наборе данных

- Название, источник, дата получения.
- Объем (строк, столбцов).
- Формат хранения (CSV, Parquet, SQL).
- Контекст (что измеряли, зачем).

Описание переменных (паспорт признаков)

- Название поля (**column name**)
- Тип (**int, float, object, datetime, bool**)
- Краткое описание
- Единицы измерения
- Диапазон допустимых значений
- Наличие и доля пропусков
- Количество уникальных значений
- Пример значений
- Замечания (аномалии, кодировки)

Технические характеристики

- Размер файла / таблицы.
- Кодировка (UTF-8, Windows-1251 и т.д.).
- Локаль / формат даты.
- Тип базы данных (если SQL).

Качество данных (Data Quality Indicators)

- Пропуски, дубли, типовые ошибки.
- Кол-во выбросов.
- Согласованность категорий.
- Доля нарушений диапазонов.

Метаданные времени

- Когда данные обновлены.
- За какой период они собраны.
- Как часто обновляются (daily, weekly, monthly).

Переходим к практике

